

The Continuous Mind: Inference-Paradigm Selection by Operator Slack Profile

A position paper, with preliminary single-GPU evidence for the always-resident personal model

Bishwanath Bastola, Independent Researcher

June 2026 · Preprint · Licensed CC-BY 4.0 · DOI: [10.5281/zenodo.20806280](https://doi.org/10.5281/zenodo.20806280)

Cite as: B. Bastola, *The Continuous Mind: Inference-Paradigm Selection by Operator Slack Profile*, Zenodo, 2026. <https://doi.org/10.5281/zenodo.20806280>

ABSTRACT. Modern LLM inference is overwhelmingly *autoregressive* (one token per sequential pass), *feed-forward* (fixed depth per call), and *request-response* (each request a cold transaction over a stateless model). We argue these are three independent **ASSUMPTIONS**, not laws, and that most operational pain — cold-start, the growing KV cache, the per-token latency floor — descends from them rather than from physics. Our central contribution is a selection principle: because every alternative paradigm *relocates* cost rather than deleting it, the right paradigm for a deployment is the one whose residual cost lands where the operator has **SLACK**. We use this to motivate one under-explored design point — the *always-resident personal model*, whose own fixed-size recurrent state is the unit of persistence and whose idle compute consolidates experience into weights — and report preliminary single-GPU evidence ($\leq 1.5\text{B}$ parameters). A tokenizer-fair bits-per-byte baseline shows the recurrent state is a strong *lossy long-range memory*: it retains a fact planted at the start of a context out to several thousand tokens, decisively beats any equal-footprint sliding window, and matches a lossless full-context Transformer's recall (no statistically significant difference over $n=10$ themes) at 13–26 $\times$ less resume memory than even a quantized (INT4/INT8) KV cache — a multiplier that grows linearly with context. A consolidation test shows idle gradient steps move a novel fact into the weights (closing $\sim 78\%$ of the gap to in-context recall) at a sweet spot, then catastrophically forget under naive full fine-tuning — motivating the standard guardrails (LoRA, low learning rate, eval-gating, rehearsal). We state confounds plainly and give a falsifiable program for converting these demonstrations into stronger evidence.

1. Introduction

The usual obstacles to on-device and distributed inference — warm-up latency, inter-shard bandwidth, the linearly-growing key-value (KV) cache, the ~ 100 tok/s decode ceiling — are treated as fixed costs. Most are not. They are consequences of three rarely-named assumptions: (1) **autoregression** — emit token t only after $t-1$, so N tokens require N non-overlapping sequential passes; (2) **feed-forward depth** — every call pays full depth; (3) **request-response** — each request is a cold transaction, and "memory" is rebuilt each time from the token history.

We separate inference issues into three tiers, because a different paradigm can only escape one of them:

Tier	Examples	Escapable by a different paradigm?
Physics floor	speed-of-light RTT, memory-bandwidth wall, energy per bit	No — only <i>avoidable</i> (compute locally, move data less)
Engineering waste	cold-start, network doglegs, recompute-per-call	Yes — relocatable
Paradigm assumption	the three above	Yes — the real lever

Almost none of the operational pain is the physics floor; it is paradigm. **Contributions.** (1) A selection principle — *inference-paradigm choice by operator slack profile* — and the no-free-lunch argument behind it (§3); this is our primary and most defensible contribution. (2) A narrowed characterization of the always-resident personal model and how it differs from concurrent work (§2, §6). (3) Preliminary single-GPU probes of its component behaviors, with confounds stated and decisive baselines identified (§4–5, §7–8).

2. Related work and the narrowed gap

Diffusion language models generate a block and refine it over a fixed number of steps, decoupling step-count from length; recent work accelerates their decoding [8–10]. **State-space / recurrent models** carry a fixed-size hidden state forward [11,12]. **Distributed inference** has chiefly split one model and shipped activations [5–7].

Most directly, several *concurrent* systems occupy parts of our design point and must be distinguished. Stateful streaming Transformers [1] invert request–response — state persists across sessions, advanced by ingesting only new tokens — and spend idle GPU cycles pre-evaluating registered queries; but their persistence is a *KV cache* (the $O(\text{context})$ growth we seek to avoid), in a datacenter setting, and idle work is query pre-evaluation, not self-consolidation. Sleep-time compute [2] does useful work while idle, but stores the result as re-represented *external context* over a stateless model. Stateful-agent frameworks (MemGPT/Letta) [13] keep the model stateless and self-manage an external paged memory. Execution-state capsules [3] checkpoint recurrent state across process death for fast cold-start, not continuous residence. LLM-as-RNN [4] frames an LLM recurrently, but its "state" is natural-language text in the prompt, not the hidden state.

***The narrowed gap.** None of these occupy the conjunction we study: a model whose own fixed-size recurrent state is the unit of persistence (not a growing KV cache, not external text), kept continuously resident on a personal always-on node, where the idle workload is the model advancing its own weights (consolidation), not pre-evaluating queries. We do not claim the components are new — state-across-process-death [3], continuous-plus-idle inference [1], and idle useful work [2] are concurrent. We claim the synthesis, and the principle (§3) that selects it.*

3. The slack-profile principle (primary contribution)

Every alternative paradigm relocates cost; none deletes it. Diffusion trades more parallel FLOPs per step for fewer sequential steps. State-space models trade lossless recall for a fixed-size (lossy) state. Analog / in-memory compute trades precision for bandwidth. Always-on operation trades idle power for zero cold-start.

***Principle.** Since residual cost can only be moved, not removed, the right paradigm for a deployment is the one whose residual lands where the operator has slack.*

A personal, always-on node has an unusual slack profile: (a) idle GPU time, since the operator wants the model resident anyway; (b) tolerance for higher per-token compute if it lowers latency-to-a-complete-response; (c) a model meant to keep running for background work regardless. This profile selects for a paradigm that keeps state resident (avoiding re-derivation) and spends spare parallel compute to soften the sequential token floor — the always-resident model of §6. We are not aware of a prior statement of paradigm-selection-by-slack-profile as a named principle; it is the cleanest novel element of this work.

4. Method: preliminary probes (and their limits)

All probes run on a single consumer GPU (NVIDIA RTX 3060, 12 GB, CUDA 12.4; PyTorch 2.6, Transformers 5.12) — a single machine, single run; results are not yet replicated across hardware or seeds. Models: RWKV-4 (169M, 1.5B) and Mamba-1 (1.4B) on the recurrent side; EleutherAI Pythia (160M, 1.4B) as nearest-size Transformers. The Mamba and Pythia runs share the GPT-NeoX-20B tokenizer; cross-model recall is reported in **bits-per-byte** (bpb), which normalizes by bytes and is therefore comparable across tokenizers. The probe suite is released with this paper.

- **P1 — does state survive process death?** Feed a context into the recurrent state, serialize it, exit the process; a fresh process loads only the state and continues. The mechanism (state survives, no replay) is the claim.
- **P1b — resume footprint vs context length.** Bytes required to resume: SSM state vs a Transformer KV cache.
- **P1.5 — the resident loop.** One long-lived process streams turns (new tokens only), self-ticks while idle, then is probed for recall.
- **P0 — decode latency shape.** Autoregression as N sequential cached passes vs a K -pass full-block compute pattern (latency shape only; not a trained diffusion model).
- **P2 / P2b / P2d — the honest baseline.** Recall in bpb against the real competitors: a Transformer with the full context in-window / KV-cached, and a sliding window, with the queried fact planted at the start of a long context.
- **P3 — consolidation.** Idle gradient steps on a novel fact, then recall a paraphrase with the fact absent from context.

5. Results (preliminary)

5.1 Persistence is the state, and it is flat

Loading only a small state blob in a fresh process reproduces the context (P1): held-out continuation loss improves by ~ 2.07 nats/token at 1.5B with zero context replay. That resume-state is flat in context length while a Transformer's KV cache grows linearly (P1b, Fig. 1): the SSM state is 0.15–0.79 MB across model scales and context lengths, while the KV cache reaches 151 MB at 4096 tokens (160M-class) and 403 MB at 2048 tokens (1.4B-class). The state is *lossy*, the KV cache is *lossless*; the fair statement is a recall-vs-memory Pareto (§5.3), not a bare flat-vs-linear plot.

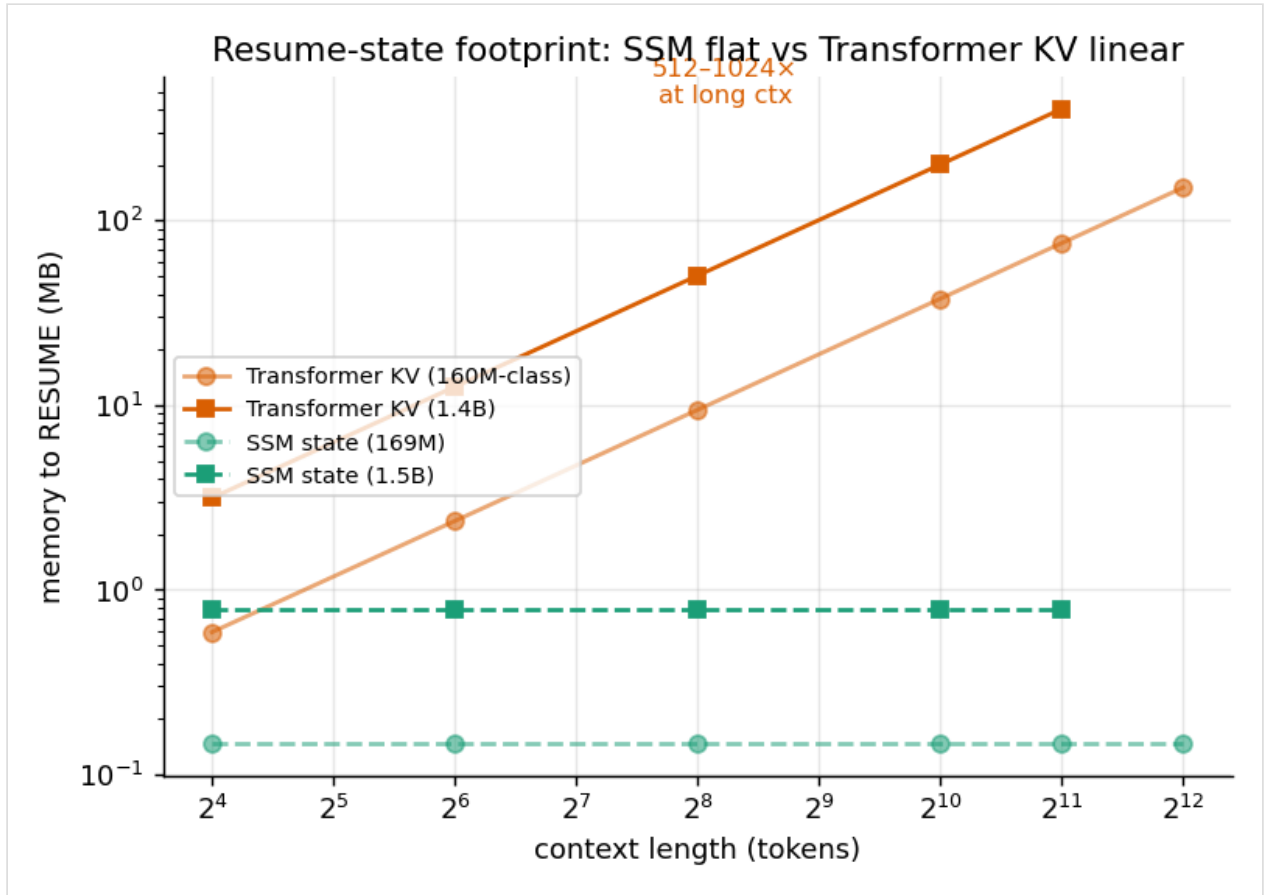


Figure 1. Resume-state footprint vs context length. The SSM state is flat; the Transformer KV cache grows linearly (512–1024× at long context). Log–log axes.

5.2 The resident loop: flat per-turn cost

A single long-lived process streaming turns processes only the new tokens each turn, giving flat per-turn latency (~21 ms at 169M, ~45 ms at 1.5B), versus a baseline that re-feeds the full history (Fig. 2). We note honestly that a production stateless server would KV-cache rather than re-prefill, giving comparable per-turn *latency*; the SSM's genuine advantage is the flat *memory* footprint (§5.3), and the resident loop also keeps an idle "dream" rollout on-theme, though we make no quality claim about it.

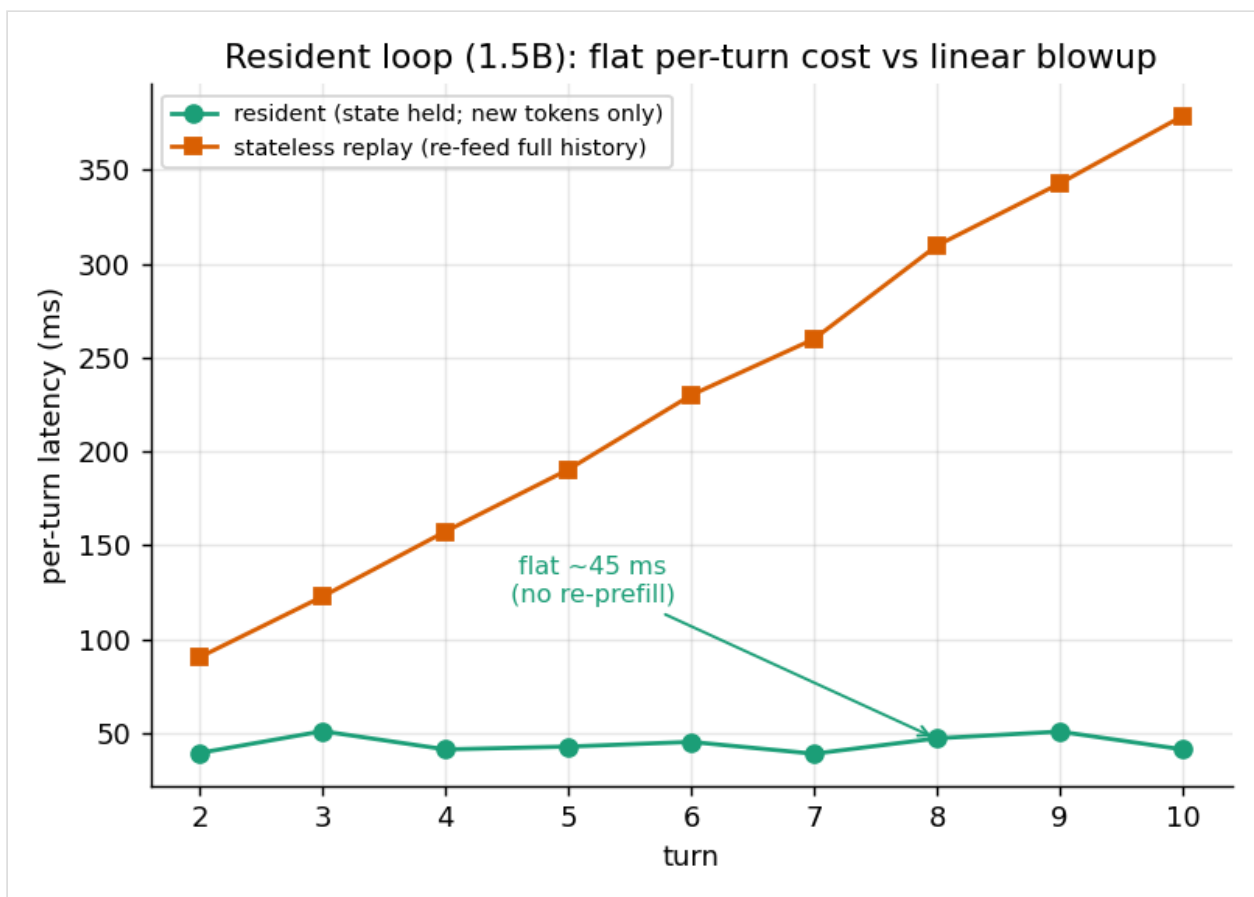


Figure 2. Resident loop (1.5B): per-turn latency stays flat (process only the new tokens) while a full-replay baseline climbs linearly.

5.3 The honest baseline: iso-recall at a fraction of, and flat, memory

Measuring recall of a held-out continuation in bits-per-byte against the real competitors (P2): the resident fixed-size state recalls *exactly as well as the same model with the full context in-window* ($0.675 = 0.675$ bpb) and ties a full-context Transformer (0.635), i.e. it has no recall *advantage* over the honest competitor — while using a flat ~ 0.79 MB vs a KV cache that already costs $12\times$ that at ~ 50 -token contexts. Pushing context length (P2b, Fig. 5; P2d, Fig. 6) is decisive: with a fact planted at the start, an equal-footprint sliding window collapses to amnesia the moment the context exceeds it, while the recurrent state retains the fact — RWKV-4-1.5B holds flat at ~ 0.8 bpb to 1024 tokens, and Mamba-1.4B (7.86 MB fixed state) holds strongly to $\sim 4k$ tokens, fades at 8k (a finite, several-thousand-token horizon), and dominates every fixed window throughout. The full-context Transformer is the only competitor that keeps pace on recall, at one to two orders of magnitude more (linearly-growing) memory; we quantify this fairly against quantized KV next (Fig. 8).

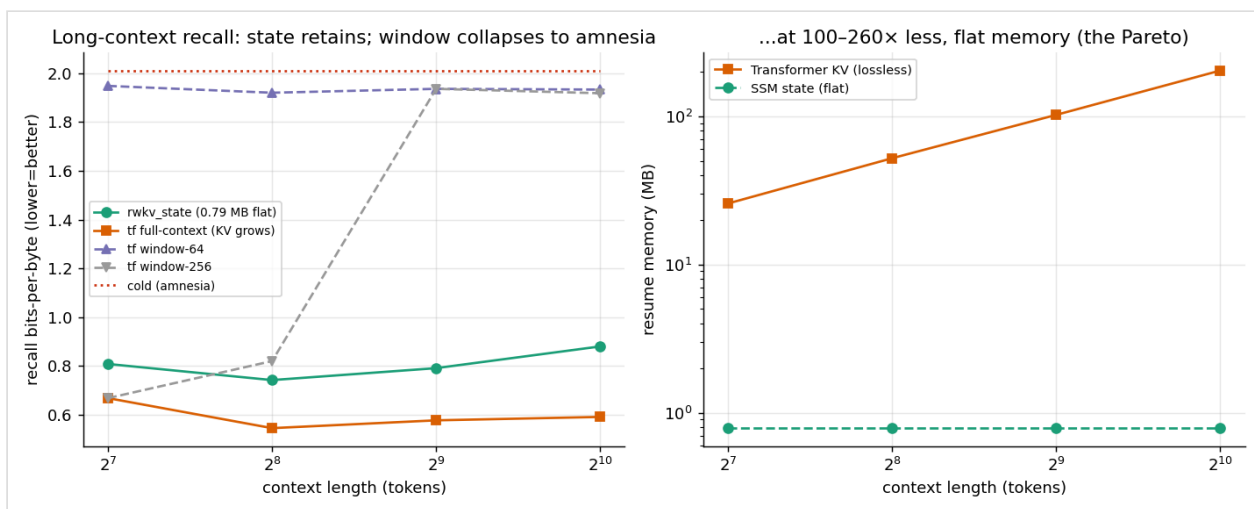


Figure 5. Long-context recall (RWKV-4-1.5B) and the memory Pareto. Left: the state holds recall while equal-footprint windows collapse to amnesia. Right: it does so at 100–260× less than an fp16 KV cache (13–26× vs quantized KV; see Fig. 8), flat.

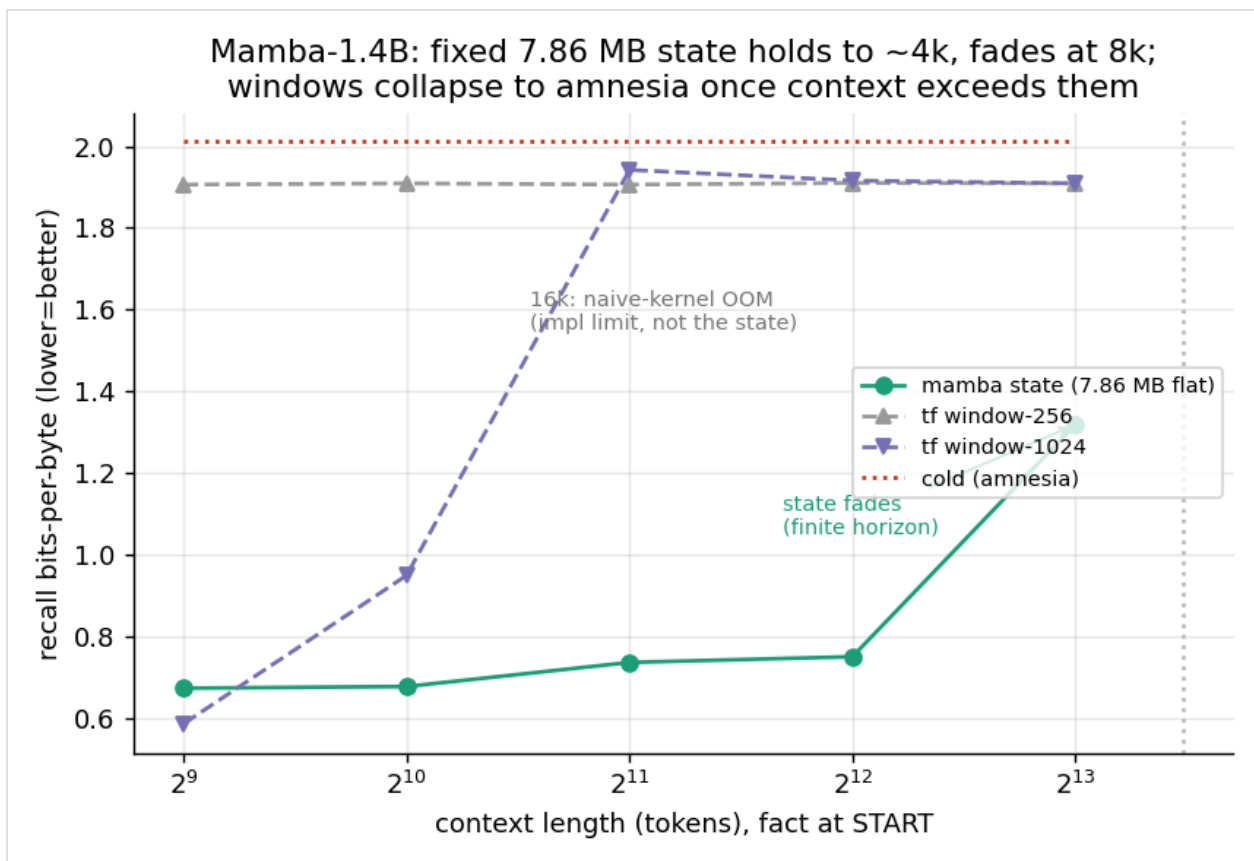


Figure 6. Mamba-1.4B to 8k tokens (fact at start). The fixed 7.86 MB state holds to ~4k and fades at 8k (finite horizon); fixed windows collapse to amnesia once context exceeds them. 16k exceeded the kernel-less scan's activation memory, an implementation limit, not the state.

Statistics and a fair, quantization-aware Pareto (P4, Fig. 8). Over $n=10$ novel fact themes with 95% confidence intervals (Mamba-1.4B, fact at start), at 2048 tokens the state scores 0.672 ± 0.158 bpb versus the full-context Transformer's 0.584 ± 0.094 — *overlapping intervals, so the recall difference is not statistically significant* — while both sliding windows have collapsed to amnesia (2.03–2.06 vs cold 2.12, far outside the state's interval). Crucially, the memory comparison is reported against *quantized* KV, the realistic serving baseline: at 2048 tokens the state's 7.86 MB is **26× smaller than an INT8 (near-lossless) KV cache** and **13× smaller than an aggressive INT4 cache** — and that multiplier grows linearly with context, since the KV

cache grows while the state is flat. The state is therefore Pareto-optimal: no system achieves lower bits-per-byte at lower resume memory.

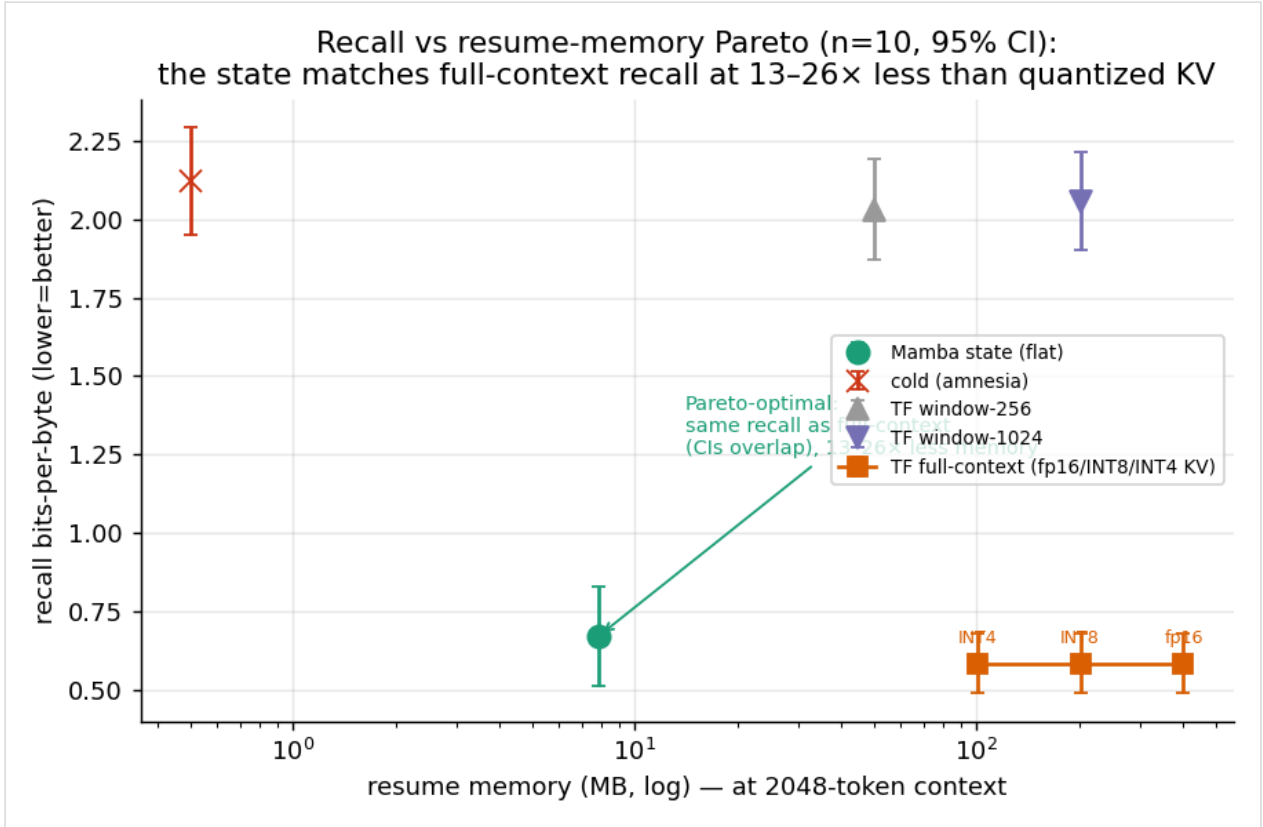


Figure 8. Recall vs resume-memory at 2048 tokens ($n=10$, 95% CI). The recurrent state matches full-context recall (overlapping intervals) at 13–26 $\times$ less memory than even a quantized KV cache; equal-footprint windows collapse to amnesia. INT4 recall is shown at the full-context value as an optimistic bound (aggressive quantization itself costs some recall).

The honest framing of this entire branch: the recurrent state is a strong *lossy long-range memory* — it matches full-context recall (within CI) at an order of magnitude less memory than even quantized KV, and dominates any equal-footprint window — rather than beating an unbounded-memory Transformer on recall quality. (Fig. 4 shows the within-model recall gap over amnesia growing with scale, $4.6\times \rightarrow 10\times$ in perplexity, reported as a within-model effect.)

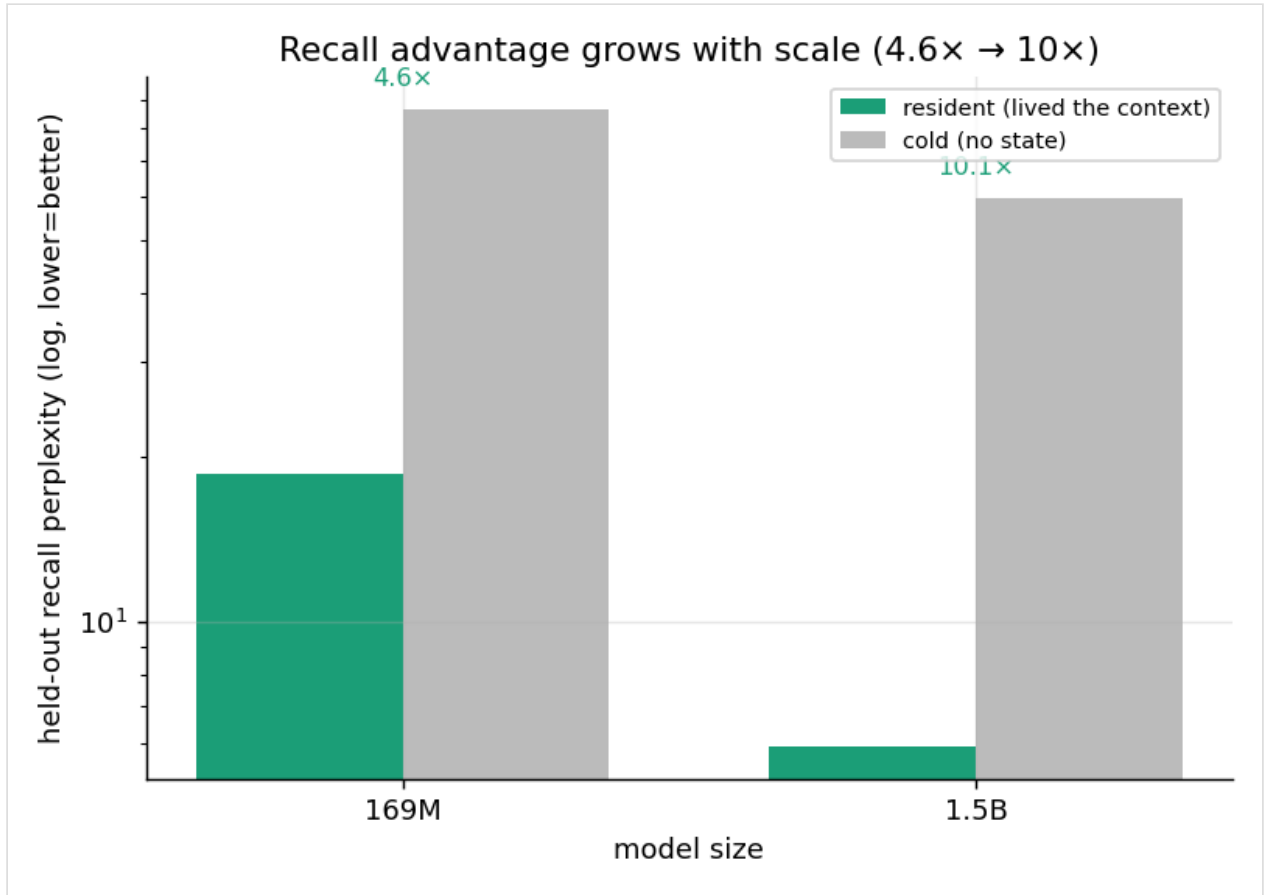


Figure 4. Within-model recall advantage over amnesia (held-out perplexity, resident vs cold), 169M vs 1.5B.

5.4 Decode latency shape

Autoregressive wall-clock is linear in output length (308 ms → 5338 ms over $N=32 \rightarrow 512$) at ~ 100 tok/s, with N sequential round-trips (P0, Fig. 3). A K -pass full-block compute pattern uses a fixed K , parallel over the block: at $N=512$, $K=16$ is $32\times$ fewer sequential round-trips. We do not report this as a diffusion speedup — the win is conditional on a K only a trained diffusion model can set, and it trades more total parallel FLOPs for fewer sequential dependencies, which is precisely the slack a personal always-on GPU has. The honest takeaway is the latency *shape*: sequential round-trips drop from N to a fixed K .

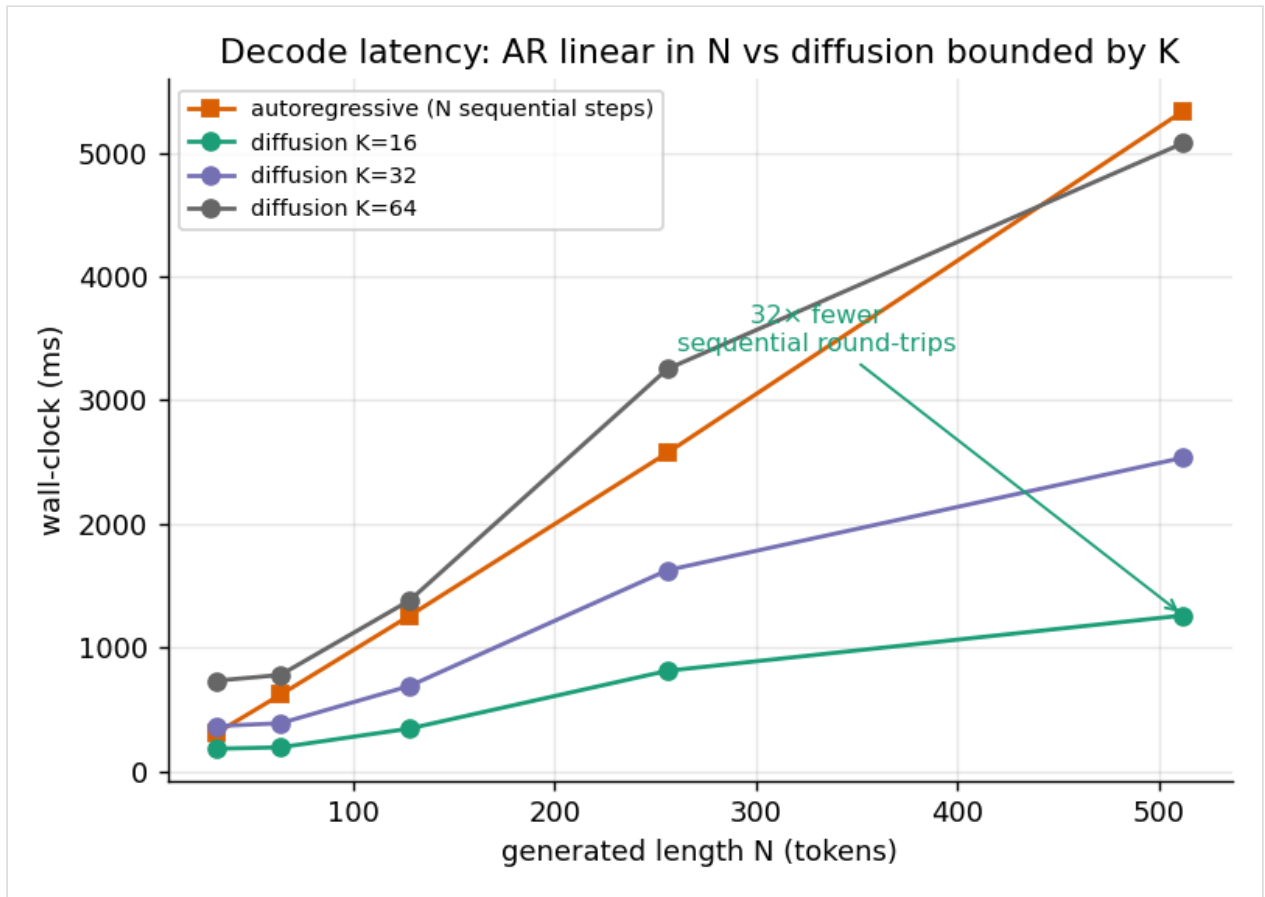


Figure 3. Decode latency shape: autoregression linear in N ; the diffusion-style compute pattern bounded by a fixed K .

5.5 Consolidation: idle work into weights, with a forgetting cliff

For a *frozen* model, idle state-rollout ("dreaming") adds no information; consolidation that improves recall must be a *weight* update (episodic $\rightarrow$ semantic). Planting novel facts and doing idle gradient steps on them, then recalling a paraphrase with the fact absent from context (P3, Fig. 7): by ~ 2 steps, cold recall improves from 2.43 to 1.27 bpb, closing $\sim 78\%$ of the gap to the in-context bound (0.93) — the fact moved into the weights. But naive full fine-tuning then catastrophically forgets (an unrelated control sentence degrades from 1.18 to 4.72 bpb). The honest conclusion: idle consolidation into weights is real *at a sweet spot*, and the well-known forgetting cliff is exactly why it must be done with low-rank adaptation, a low learning rate, an eval-gate that rejects bad checkpoints, and rehearsal.

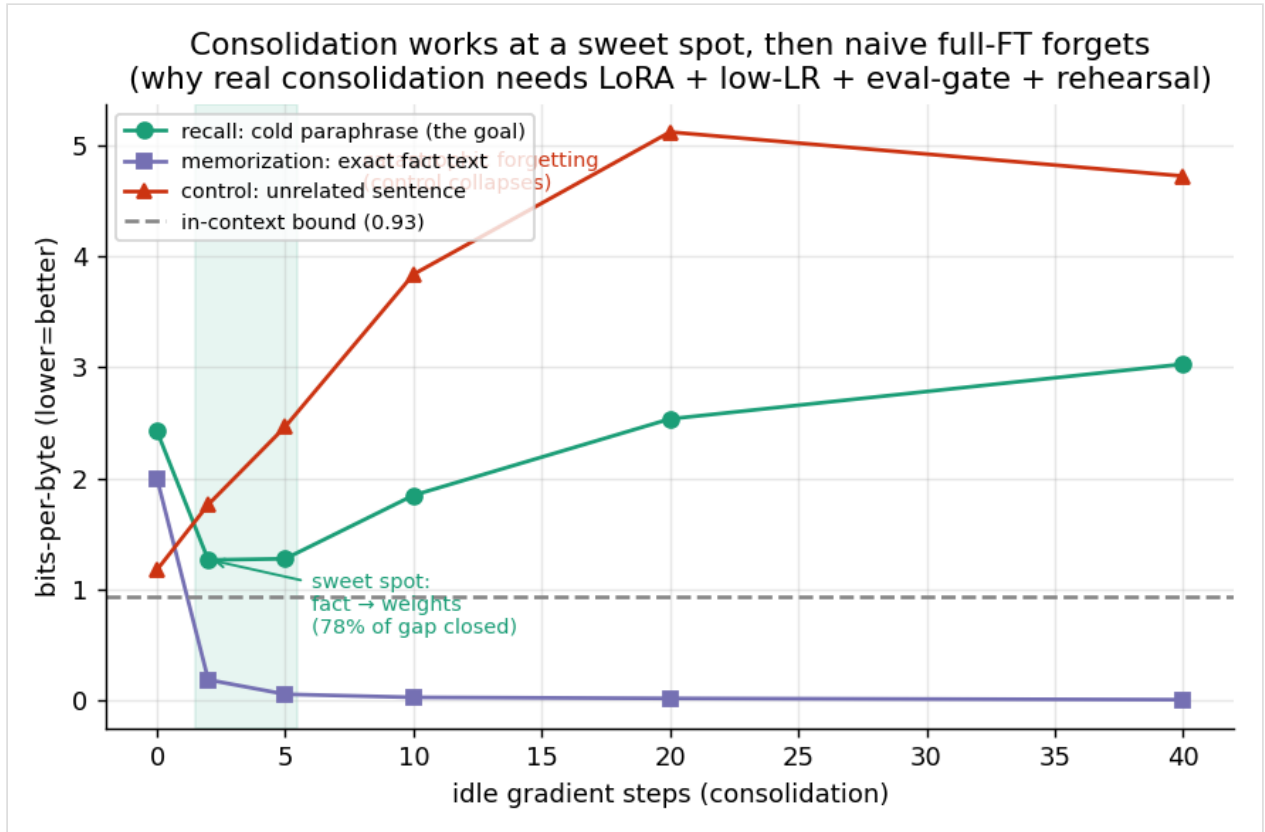


Figure 7. Consolidation (Pythia-160M, novel facts). Idle gradient steps move a fact into the weights at a sweet spot (~78% of the gap to in-context closed), then naive full fine-tuning catastrophically forgets (control collapses).

6. The Continuous Mind (proposed architecture)

The probes motivate, but do not yet validate, a single architecture: run one model as an always-resident dynamical system — recurrent/SSM state as fixed-size persistence (P1, P1b), spare idle parallel compute to soften the sequential token floor (P0), and idle time spent consolidating recent experience into the weights (P3 — a guarded low-rank update, not free state-rollout). It is steered occasionally and runs always. It differs from concurrent stateful-streaming [1] (KV cache, datacenter, query pre-evaluation) and sleep-time compute [2] (external text over a stateless model) precisely in using the model's *own* fixed-size state as persistence and idle time for *self*-consolidation on a personal node. Whether this delivers a net advantage over a well-engineered KV-cached Transformer is an open empirical question (§8), not a settled result.

7. Limitations (what is *not* shown)

- **Scale.** ≤1.5B parameters on a single 12 GB GPU, single run, no seeds or variance estimates, few prompts per probe.
- **The state is lossy and saturating.** Mamba-1.4B's fixed state fades by ~8k tokens; long-range recall has a finite horizon, not unbounded memory.
- **No downstream task benchmark.** Recall is measured by held-out bits-per-byte, not task accuracy; quality parity with an equal-size Transformer is not established.
- **P0 is not diffusion.** It measures a compute pattern on a causal model; no diffusion latency or text quality is established, and the minimum K a trained dLLM needs is unmeasured.
- **Consolidation needs guardrails.** Demonstrated at small scale; naive full fine-tuning forgets — the guarded (LoRA + eval-gate) version is future work.
- **Always-warm is node-local.** Zero cold-start applies to the operator's own resident node, not to renting third-party GPUs.

8. Future work (the decisive experiments)

1. **Long-context Pareto at scale** with fused-kernel SSMs (Mamba-2, RWKV-7) to $\geq 16k$ tokens, where even a quantized KV cache becomes prohibitive.
2. **Recall-vs-resume-bytes Pareto** against a quantized/paged KV cache, the realistic lossless baseline.
3. **Guarded consolidation** (LoRA + eval-gate + rehearsal) at $\geq 1B$, showing the sweet spot held without the forgetting cliff.
4. **Trained diffusion LM** to convert P0's latency shape into an absolute quality-vs- K number.
5. **Statistics and scale** — multiple prompts and seeds with confidence intervals, ≥ 3 model sizes before any scaling-trend claim.

9. Conclusion

The bottlenecks of standard inference are largely assumptions, not physics — and the most useful lever is choosing a paradigm whose residual cost lands on the operator's slack. That selection principle is, to our knowledge, unstated in the literature, and is this paper's core contribution. The design point it selects — an always-resident model whose own fixed-size state is the persistence and whose idle time consolidates into weights — is, in its individual mechanisms, now appearing in concurrent work; the specific synthesis remains, to our knowledge, unoccupied. Our probes are preliminary single-GPU demonstrations of its component behaviors, with confounds stated plainly and decisive baselines identified. The honest status is a promising principle with a concrete, falsifiable experimental program — not a demonstrated system.

Reproducibility

The probe suite (P0–P3), figure-generation script, and raw measurement outputs are released alongside this paper. Each probe is a self-contained script with the model identifiers, metric definitions, and the exact prompts used; all figures are regenerated from the recorded numbers. Experiments were conducted with AI-assisted implementation and adversarial review of the claims prior to release.

References

1. V. Norgren. *Attention Once Is All You Need: Efficient Streaming Inference with Stateful Transformers*. arXiv:2605.13784, 2026.
2. K. Lin, C. Snell, Y. Wang, C. Packer, S. Wooders, I. Stoica, J. E. Gonzalez. *Sleep-time Compute: Beyond Inference Scaling at Test-time*. arXiv:2504.13171, 2025.
3. L. Su. *Execution-State Capsules: Graph-Bound Execution-State Checkpoint and Restore for Low-Latency, Small-Batch, On-Device Physical-AI Serving*. arXiv:2606.20537, 2026.
4. Y. Lu, J. B. Tamo, W. Zhao, N. Sun, Y. Zhong, W. Shi, J. Wang, M. D. Wang. *LLM-as-RNN: A Recurrent Language Model for Memory Updates and Sequence Prediction*. arXiv:2601.13352, 2026.
5. A. Borzunov et al. *Petals: Collaborative Inference and Fine-tuning of Large Models*. arXiv:2209.01188, 2022.
6. *Distributed Inference and Fine-tuning of Large Language Models Over the Internet*. arXiv:2312.08361, 2023.
7. *Demystifying Platform Design for Distributed Inference of LLMs*. arXiv:2406.01698, 2024.
8. *Attention Is All You Need for KV Cache in Diffusion LLMs*. arXiv:2510.14973, 2025.
9. *Focus-DLLM: Accelerating Long-Context Diffusion-LLM Inference*. arXiv:2602.02159, 2026.
10. *DMax: Aggressive Parallel Decoding for Diffusion LLMs*. arXiv:2604.08302, 2026.
11. *State-Space Modeling in Long-Sequence Processing: A Survey on Recurrence in the Transformer Era*. arXiv:2406.09062, 2024.
12. *RotRNN: Modelling Long Sequences with Rotations*. arXiv:2407.07239, 2024.
13. C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.